

DIVIDE AND CONQUER – SEARCHING & BASIC PROBLEMS

Binary Search + Maximum and Minimum

QUESTION 1: SEARCH AN ELEMENT IN A SORTED ARRAY

```
def binary_search_q1(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            print("Element found at position", mid + 1)
            return

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    print("Element not found")

print("\n===== QUESTION 1 - TEST CASE 1 =====")
binary_search_q1([10, 20, 30, 40, 50], 30)

print("\n===== QUESTION 1 - TEST CASE 2 =====")
binary_search_q1([10, 20, 30, 40, 50], 25)
```

```
===== QUESTION 1 - TEST CASE 1 =====
Element found at position 3

===== QUESTION 1 - TEST CASE 2 =====
Element not found
```

QUESTION 2: FIND THE INDEX OF A GIVEN NUMBER

```
def binary_search_q2(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
```

```

        if arr[mid] == target:
            return mid

        elif arr[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1

print("\n===== QUESTION 2 - TEST CASE 1 =====")
arr = [2, 4, 6, 8, 10, 12]
target = 8

index = binary_search_q2(arr, target)
print("Index =", index)

print("\n===== QUESTION 2 - TEST CASE 2 =====")
arr = [2, 4, 6, 8, 10, 12]
target = 5

index = binary_search_q2(arr, target)
print("Index =", index)

```

```

===== QUESTION 2 - TEST CASE 1 =====
Index = 3

===== QUESTION 2 - TEST CASE 2 =====
Index = -1

```

✓ QUESTION 3: COUNT NUMBER OF ITERATIONS

```

def binary_search_iterations(arr, key):
    low = 0
    high = len(arr) - 1
    iterations = 0

    while low <= high:

        iterations += 1

        mid = (low + high) // 2

        if arr[mid] == key:
            print("Element found")
            print("Iterations =", iterations)
            return

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

```

```

    print("Element not found")
    print("Iterations =", iterations)

print("\n===== QUESTION 3 - TEST CASE 1 =====")
binary_search_iterations(
    [5, 10, 15, 20, 25, 30, 35],
    25
)

print("\n===== QUESTION 3 - TEST CASE 2 =====")
binary_search_iterations(
    [5, 10, 15, 20, 25, 30, 35],
    40
)

```

```

===== QUESTION 3 - TEST CASE 1 =====
Element found
Iterations = 3

===== QUESTION 3 - TEST CASE 2 =====
Element not found
Iterations = 3

```

✓ QUESTION 4: FIRST OCCURRENCE

```

def first_occurrence(arr, key):
    low = 0
    high = len(arr) - 1
    result = -1

    while low <= high:

        mid = (low + high) // 2

        if arr[mid] == key:
            result = mid
            high = mid - 1

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return result

print("\n===== QUESTION 4 - TEST CASE 1 =====")

arr = [1, 2, 2, 2, 3, 4, 5, 6]
key = 2

index = first_occurrence(arr, key)

if index != -1:

```

```

    print("First occurrence at index", index)
elif index == -1:
    print("Element not found")

print("\n===== QUESTION 4 - TEST CASE 2 =====")

arr = [1, 2, 2, 2, 3, 4, 5, 6]
key = 7

index = first_occurrence(arr, key)

if index != -1:
    print("First occurrence at index", index)
elif index == -1:
    print("Element not found")

===== QUESTION 4 - TEST CASE 1 =====
First occurrence at index 1

===== QUESTION 4 - TEST CASE 2 =====
Element not found

```

✓ QUESTION 5: LAST OCCURRENCE

```

def last_occurrence(arr, key):
    low = 0
    high = len(arr) - 1
    result = -1

    while low <= high:

        mid = (low + high) // 2

        if arr[mid] == key:
            result = mid
            low = mid + 1

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return result

print("\n===== QUESTION 5 - TEST CASE 1 =====")

arr = [1, 2, 2, 2, 3, 4, 5, 6]
key = 2

index = last_occurrence(arr, key)

if index != -1:
    print("Last occurrence at index", index)

```

```

elif index == -1:
    print("Element not found")

print("\n===== QUESTION 5 - TEST CASE 2 =====")

arr = [1, 2, 2, 2, 3, 4, 5, 6]
key = 9

index = last_occurrence(arr, key)

if index != -1:
    print("Last occurrence at index", index)
elif index == -1:
    print("Element not found")

===== QUESTION 5 - TEST CASE 1 =====
Last occurrence at index 3

===== QUESTION 5 - TEST CASE 2 =====
Element not found

```

✓ DIVIDE AND CONQUER MINIMUM AND MAXIMUM

```

def find_min_max(arr, low, high):

    # Only one element
    if low == high:
        return arr[low], arr[low]

    # Two elements
    if high == low + 1:

        if arr[low] < arr[high]:
            return arr[low], arr[high]
        else:
            return arr[high], arr[low]

    # Divide
    mid = (low + high) // 2

    left_min, left_max = find_min_max(
        arr,
        low,
        mid
    )

    right_min, right_max = find_min_max(
        arr,
        mid + 1,
        high
    )

    # Conquer
    minimum = min(left_min, right_min)
    maximum = max(left_max, right_max)

```

```
return minimum, maximum
```

QUESTION 6: HIGHEST AND LOWEST TEMPERATURE

```
print("\n===== QUESTION 6 - TEMPERATURE =====")

temperatures = [32, 28, 35, 25, 31, 29, 37]

minimum, maximum = find_min_max(
    temperatures,
    0,
    len(temperatures) - 1
)

print("Minimum Temperature =", minimum)
print("Maximum Temperature =", maximum)
```

```
===== QUESTION 6 - TEMPERATURE =====
Minimum Temperature = 25
Maximum Temperature = 37
```

QUESTION 7: BEST AND WORST STUDENT MARKS

```
print("\n===== QUESTION 7 - STUDENT MARKS =====")

marks = [78, 92, 65, 88, 95, 72, 81, 69]

minimum, maximum = find_min_max(
    marks,
    0,
    len(marks) - 1
)

print("Minimum Mark =", minimum)
print("Maximum Mark =", maximum)
```

```
===== QUESTION 7 - STUDENT MARKS =====
Minimum Mark = 65
Maximum Mark = 95
```

QUESTION 8: STOCK PRICE ANALYSIS

```
print("\n===== QUESTION 8 - STOCK PRICE =====")

prices = [450, 520, 480, 610, 430, 590]

minimum, maximum = find_min_max(
    prices,
    0,
    len(prices) - 1
)
```

```
print("Minimum Stock Price =", minimum)
print("Maximum Stock Price =", maximum)
```

```
===== QUESTION 8 - STOCK PRICE =====
Minimum Stock Price = 430
Maximum Stock Price = 610
```

✓ QUESTION 9: FASTEST AND SLOWEST RACE TIMES

```
print("\n===== QUESTION 9 - RACE TIMES =====")

times = [14, 12, 18, 11, 15, 13, 17, 16, 10]

minimum, maximum = find_min_max(
    times,
    0,
    len(times) - 1
)

print("Fastest Time =", minimum)
print("Slowest Time =", maximum)
```

```
===== QUESTION 9 - RACE TIMES =====
Fastest Time = 10
Slowest Time = 18
```

✓ QUESTION 10: LARGEST AND SMALLEST BUILDING HEIGHTS

```
print("\n===== QUESTION 10 - BUILDING HEIGHTS =====")

heights = [
    120, 150, 98, 175, 140,
    165, 110, 190, 130, 145
]

minimum, maximum = find_min_max(
    heights,
    0,
    len(heights) - 1
)

print("Minimum Height =", minimum)
print("Maximum Height =", maximum)
```

```
===== QUESTION 10 - BUILDING HEIGHTS =====
Minimum Height = 98
Maximum Height = 190
```

✓ COMPLEXITY ANALYSIS

```
print("\n=====")
print("COMPLEXITY ANALYSIS")
print("=====")

print("\nBinary Search:")
print("Best Case      : O(1)")
print("Average Case   : O(log n)")
print("Worst Case      : O(log n)")
print("Space           : O(1) for iterative version")

print("\nFirst/Last Occurrence using Binary Search:")
print("Time            : O(log n)")
print("Space           : O(1)")

print("\nFinding Minimum and Maximum using Divide and Conquer:")
print("Time            : O(n)")
print("Space           : O(log n) due to recursion")
```

```
=====
COMPLEXITY ANALYSIS
=====

Binary Search:
Best Case      : O(1)
Average Case   : O(log n)
Worst Case     : O(log n)
Space          : O(1) for iterative version

First/Last Occurrence using Binary Search:
Time           : O(log n)
Space          : O(1)

Finding Minimum and Maximum using Divide and Conquer:
Time           : O(n)
Space          : O(log n) due to recursion
```